

ROS2 Day4 hw1 과제 보고서

로봇 21기 예비단원 최윤서

1. Lifecycle이란?

1-1. 개념

- ROS2에서 노드 상태를 관리하기 위해 제공하는 인터페이스
- 노드의 생명주기를 명확하게 관리 가능
- 개발자가 노드의 상태와 동작을 더 세밀하게 제어할 수 있도록 함
→ 복잡한 로봇 시스템에서 자원관리와 예측 가능한 동작을 가능케함

1-2. Lifecycle 상태 전환 과정

1-2-1. 주요 상태

1) Unfigured

- 노드가 생성된 직후의 상태
- 재구성 가능, 변수들이 형식적으로만 선언

2) Inactive

- 구성 완료된 상태
- 파라미터 설정, 토픽 구성 가능
- 실제 데이터는 처리 X

3) active

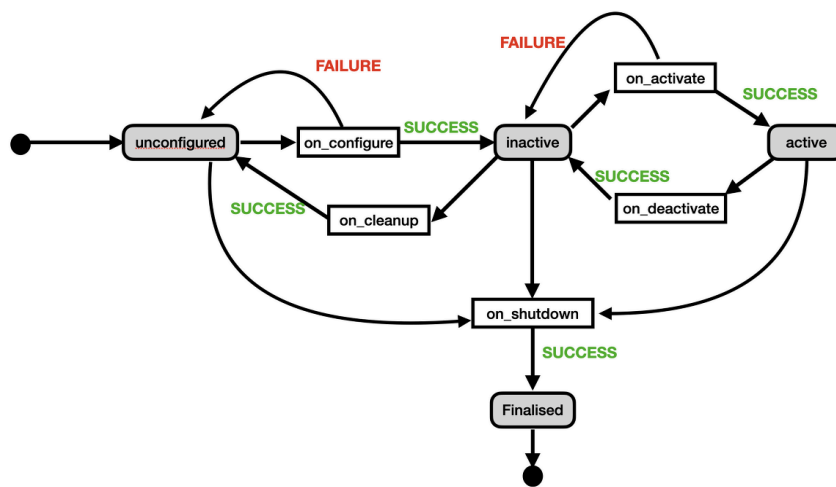
- 노드가 실제로 동작 수행하는 단계
- 모든 기능 활성화

4) finalized

- 노드 종료 전 상태
- if 에러 발생 → 디버깅 용으로 사용
- 리소스 정리 역할 수행

1-2-2. 전환 상태

- 1) configuring: 노드 설정 수행 중
- 2) Activating: 동작 시작 전 준비 단계
- 3) Deactivating: 동작 멈춤 → Inactivate로 전환
- 4) cleaning up: 내부 구성 초기화(재설정 전에 호출)
- 5) Shutting Down: 노드 종료 준비(메모리 해제)
- 6) ErrorProcessing: 에러 발생 시 처리



2. Lifecycle을 사용해야하는 이유

- ROS2에서 일반 노드는 실행되자마자 동작을 시작 → 발행과 구독이 실행되면서 메시지 송수신 시작
- 하지만, 실제 로봇을 작동시킬 때 어떠한 준비 단계를 거치고 노드를 실행시키고싶거나 하드웨어 장치 초기화 전까지는 동작을 하지 않아야한다는 등 노드를 실행시키기 전에 필요한 과정이 있을 수 있음
- 따라서, 체계적이고 상태 전환을 통해 단계를 설정할 수 있는 Lifecycle 인터페이스를 사용해야함

3. Lifecycle을 구현하기에 적합한 패키지

- rclcpp_lifecycle / lifecycle_msgs 등이 Lifecycle을 구현하기에 적합
- 근거: 2번에서 말했던 Lifecycle의 장점처럼 노드의 초기화, 활성화, 비활성화, 종료 과정을 단계적으로 관리할 수 있고 위의 패키지들은 그렇게 단계적으로 나뉘어서 실제 로봇 시스템에 적용시켜야 오류가 생겼을 때 어느 단계에서 생겼는지 정확히 알 수 있음

- 게다가 rclcpp_lifecycle은 로봇 제어 및 센서와 같이 안정적인 상태 관리가 필요한 시스템에 적합하기 때문이다.

4. QoS란?

4-1. 개념

- Quality of Service의 약자로 네트워크에서 트래픽을 우선순위에 따라 분류, 관리하는 시스템

4-2. 주요 요소

1) 대역폭

- 데이터 전송을 위해 사용할 수 있는 네트워크 용량
- QoS는 특정 데이터 흐름에 필요한 대역폭 보장 → 안정적인 서비스 제공

2) 지연

- 데이터가 출발지에서 목적지로 전달되는 데 걸리는 시간
- 지연이 크다? → VoIP 같은 실시간 서비스 품질 저하

3) 지터

- 데이터 전달 시간의 변동성을 의미
- 일정하지 않는 지터는 음성통화나 스트리밍 서버에 악영

4) 패킷 손실

- 데이터 전달 시간의 변동성을 의미
- QoS는 네트워크 혼잡 완화 → 패킷 손실 줄임, 데이터 신뢰 고정

4-3. QoS 매개변수

1) Reliability: 메시지 전달 보장 수준(Best Effort / Reliable)

- Best Effort: 데이터 유실 발생 시 저장안하고 그대로 감
- Reliable: 데이터 유실 발생 시 뒤로 다시 가서 저장하고 감

2) Durability: 과거 메시지 보존 여부(Volatile / Transient Local)

- Transient_local: 과거 메시지 보존
ex) 1,2 메시지 → DataReader 생성 → 1,2 유지

- Volatile: 과거 메시지 보존 X
ex) 1,2 메시지 → DataReader 생성 → 메시지 초기화

3) History: 버퍼 크기(Keep all / Keep Last / Depth)

- Depth: Keep Last일 때 유지할 Queue의 크기
- Keep all: 버퍼나 큐에 들어오는 모든 데이터 삭제 안하고 전부 보존
- Keep Last: 패킷의 특정 마킹 값 변경 안하고 그대로 유지

4) Dead line: 메시지 전달 데드라인 시간

5) Liveliness: 노드 생존 여부 파악 방식

5. QoS가 중요한 이유

- 과거: 각 데이터를 별도의 망으로 운영, 전화 통화도 전용 회선 사용, 컴퓨터와 서버는 별도의 데이터 네트워크로 연결
→ 네트워크 교차 경우 거의 X → 트래픽 간의 충돌 X
- 현재: 하나의 네트워크에서 영상통화, 스트리밍, 업무 데이터 동시에 흐름
→ 트래픽 간의 충돌 발생 많음
- 현재에서 네트워크 자원을 효율적으로 배분하는 핵심 역할

4. 각 노드에 적합한 QoS 설정

6-1. VoIP 트래픽 우선순위 지정

- if 기업 네트워크 혼잡 상태 → 라우터와 스위치 처리할 수 있는 것보다 더 빨리 들어오고 나갈 때 패킷이 누락되기 시작
- 그 결과, 스트리밍 애플리케이션 작동 어려워짐
- 이때 우선순위를 지정 → 유형과 대상에 따라 트래픽 분류 → 각각 다른 우선순위 배정 → 높은 우선순위를 가진 패킷이 다른 트래픽보다 먼저 전송
→ 혼잡한 상황에서 유용하게 사용

6-2. 리소스 예약

- 네트워크를 통해 리소스 예약

- 애플리케이션 데이터 스트림에 대해 특정 수준의 QoS를 제공하는 데 사용할 수 있는 전송 계층 프로토콜
- 리소스 예약을 통해 기업은 네트워크 리소스를 다양한 유형의 출처의 트래픽으로 나눔
- 제한 정의 → 대역폭 보장

6-3. 대기(Queueing)

- 다른 데이터 스트림보다 특정 데이터 스트림 우선 처리
- 대기열(Queue)은 통과하는 패킷이 전용 메모리 영역에 보관되는 라우터 및 스위치의 고성능 메모리 버퍼
- 더 높은 우선순위 할당 → 데이터 더 빠르게 푸시하는 전용 대기열로 이동
→ 삭제 가능성 줄어듦